

The LangGraph Building Blocks

A one page reference for every graph you build

Every LangGraph agent you build, from the three node research loop to a multi agent orchestration, is assembled from the same twelve pieces. Keep this card next to the notebook. When a graph does not behave, the fault is almost always in one of these, and knowing which one narrows the search fast.

Building block	What it is, and the one thing to remember
State	A TypedDict scratchpad shared by every node, carried from START to END for a single run. Declare the fields you want tracked. It is not a model and it is not memory.
Workflow state	Plain types such as string, integer and boolean. Every write overwrites the last value, so the field only ever holds the latest one.
Conversational state	A field that must accumulate, usually a list of messages. Needs a reducer, otherwise each new message replaces the whole history.
Reducer	A function that defines how a new value merges into the existing one. <code>add_messages</code> appends a single value, <code>operator .add</code> concatenates two lists, <code>max</code> keeps the highest. Write your own and pass its name in the annotation.
Node	A plain Python function. Takes the state, returns a dictionary containing only the fields it changed . The keys must be state field names as plain strings, never <code>state['field']</code> .
START and END	Reserved nodes. Every graph begins at one and finishes at the other. You never define them, you only connect to them.
Edge	An unconditional connection, added with <code>builder.add_edge(from, to)</code> . Direction matters and reads left to right. Several edges out of one node run in parallel when the targets are independent.
Conditional edge	An edge whose destination is chosen at runtime, added with <code>add_conditional_edges(source, router, mapping)</code> . The dotted lines in a graph diagram.
Router function	Reads the state and returns a string naming the route. Read only: it can never modify the state. The string is looked up in the mapping to find the destination node.
compile()	<code>builder.compile()</code> turns the description into a runnable graph. Nothing executes until you compile, and the compiled object is what you invoke.
Checkpointter	Saves a snapshot of the state after every node so a run can be resumed rather than restarted. <code>MemorySaver()</code> keeps it in RAM. Production points it at Postgres, SQLite or another database.
interrupt	Pauses the graph and hands control plus the current state back to the caller. The payload you pass is what your UI shows the human. Requires a checkpointter, or you get an error.

Building block	What it is, and the one thing to remember
Command	How you resume an interrupted graph. Invoke again with the same config, passing a Command carrying the human's response. It picks up from the checkpoint, it does not start over.
thread_id	A unique identifier inside config, the same idea as a session ID. Tells LangGraph which running graph you mean when several are live.
get_state	<code>graph.get_state(config)</code> returns a snapshot of a paused graph. <code>snapshot.next</code> tells you which node it is waiting on.
Subgraph	A compiled graph invoked from inside a node of another graph. Each keeps its own state, and you control exactly what crosses the boundary. Share only what the parent needs.
bind_tools	<code>llm.bind_tools(tools)</code> gives the model visibility of the tools it may call. Same as in LangChain.
ToolNode	Wraps a tool list so the graph can execute a tool call as a node. Inspect <code>tools_by_name</code> to confirm what is registered.
tool_calls	The field on the returned AIMessage that says whether the model wants a tool. Present means route to the tools node, empty means the answer is ready.

How to use this card

Do not start from code, start from the workflow. Sketch the boxes and arrows first: what are the steps, which connections are unconditional, and where does a decision have to be made. Only then translate the sketch into nodes, edges and conditional edges. Starting from the code is what makes LangGraph feel confusing.

The three mistakes that cost the most time

- 1. Returning the whole state from a node.** Return only the delta. LangGraph merges it.
- 2. Expecting a router to update the state.** It returns a string and nothing else. If you need the reason for a decision recorded, write it from inside a node.
- 3. Using interrupt without a checkpoint.** You told the graph to stop but gave it nowhere to record where it stopped, so there is nothing to resume from.

Workflow or agent?

The graph structure does not decide this, the nodes do. Plain deterministic Python functions with no tool calling and no autonomous action give you a **workflow application**. The moment nodes call tools and act on their own, the same graph becomes an **agentic application**. LangGraph is the conductor either way.